# Cisco ACI Upgrade Automation with Claude: Research and Content Plan

Network and Infrastructure Automation · 3 days (8 hours per day) · Intermediate to advanced · Compiled 30 October 2026

This document is the working plan for building the course from its outline. It records where the course came from, why the outline is shaped the way it is, what goes into every module (lessons, teaching points, demo, lab, pitfalls and knowledge checks), the lab environment to prepare, and the sources behind each fact. Numbers in square brackets point to the reference list at the end; every URL there was opened and checked against the fact it supports.


- 1. Where this course came from
- 2. How the outline was shaped
- 3. Audience and prerequisites
- 4. Lab environment
- 5. Module-by-module content plan
- 6. Capstone plan
- 7. Promotional video: research behind the script
- References


## 1. Where this course came from

**Client input.**

> Client use case 10, 'Cisco ACI Upgrade Automation: Managing upgrade of ACI chassis as per process suggested by Cisco', with the client's aim 'to do automation using Claude for all these content' and course names of the form 'Automation of <> using Claude'. Use case 2 (device upgrades with space clear, firmware upload, pre and post checks and a scheduler option) shaped the bootflash, staging and scheduling modules.

ACI upgrades follow a documented Cisco sequence: back up, download images, validate, update CIMC, upgrade the APIC cluster, validate again, wait for Fully Fit, then upgrade switch groups [2]. Most of the risk sits in the details: groups that put both vPC peers down at once, an unhealthy cluster, bootflash without room for the image, or an operator reloading an APIC when an upgrade stalls [3][12][15]. These are exactly the steps where checklists get skipped under weekend pressure.

The work is also well suited to automation with a clear split. Reading validator output, joining it with inventory and writing a plan and a report is language-heavy work that Claude does well, while creating policies and triggering groups must be deterministic, approved and logged. Cisco exposes every step through the APIC REST API and Ansible modules [7][10][22], and Claude's Agent SDK can return plans as schema-validated JSON [26], so the course can teach a workflow where the model never touches a write path.

The client asked for automation of ACI upgrades 'as per process suggested by Cisco'. The course therefore follows Cisco's guide chapter by chapter [1] and adds the controls an enterprise change board expects: a pull request, a ServiceNow change, window enforcement and an audit log.

## 2. How the outline was shaped

| Decision | Rationale |
|---|---|
| Follow Cisco's documented workflow as the spine of the course. | The client input says 'as per process suggested by Cisco'; mapping modules to the guide's workflow keeps the course defensible and current [1][2]. |
| Claude plans and explains; Python and Ansible execute. | Write actions on a fabric must be repeatable and reviewable; the agent has read-only MCP tools and denied write tools, enforced by permissions and hooks, not prompts [24][27][28]. |
| Use structured outputs and a code gate for the plan. | A plan that breaks a grouping rule must be rejected before a person reads it; the SDK's JSON Schema output plus Python rule checks make that testable [3][26]. |
| Put approval in both GitHub and ServiceNow. | Engineers review the plan as code in a PR with required checks, and the change board approves the window in ServiceNow; the runner checks both [32][36]. |
| Bring in the bootflash, staging and scheduler items from use case 2. | Space clearing, firmware upload, pre and post checks and scheduling were explicit client asks; on ACI they map to the bootflash check, the firmware repository and window enforcement [12][17][21]. |
| Use a small physical fabric plus a mocked APIC. | Real reboots and cluster conversion cannot be shown on the DevNet simulator, and failure paths cannot be safely forced on hardware, so both are used [37]. |

## 3. Audience and prerequisites

| Role | What they need | Starting point |
|---|---|---|
| Data center network engineer | A safe, repeatable upgrade process with evidence | Runs ACI day to day, has done GUI upgrades, little Python |
| Network automation engineer | Patterns for agents with read-only tools and code-driven execution | Python and Ansible comfortable, less ACI upgrade depth |
| NOC or change manager | Understanding of the gates and the audit output | Knows change process, light technical depth; attends days 1 and 2 selectively |
| Platform or SRE engineer supporting network teams | CI, secrets and runner design | Strong GitHub Actions, little ACI |

### Prerequisites from the outline

- Working knowledge of Cisco ACI: APIC, leaf and spine, tenants, vPC

Gemnor Tech TRAINING For more info - https://gemnor.com

- Basic Python and REST API calls with JSON
- Familiarity with change management and maintenance windows
- Familiarity with an AI coding assistant is helpful, not required

### Checks before enrolment

- Can you explain what a vPC pair is and why both peers should not reboot together?
- Have you logged in to an APIC and found a node's firmware version in the GUI?
- Can you write a Python function that calls a REST API and parses JSON?
- Do you know what a normal change and a CAB approval are in your organization?
- Do you have a GitHub account and can you open a pull request?

### Course outcomes

- Describe the Cisco-recommended ACI upgrade sequence: release selection, pre-upgrade validation, APIC cluster first, then switch groups, then post-checks
- Build a Python client for the APIC REST API that discovers the fabric, runs the pre-upgrade validator and returns typed JSON
- Use Claude Code, the Claude Agent SDK and an MCP server to plan upgrade groups and maintenance windows as schema-validated structured output
- Create firmware and maintenance policies with deterministic Python and Ansible only after a human approves the plan in a GitHub pull request and a ServiceNow change
- Produce pre and post upgrade evidence, a comparison report, a pause or rollback decision and a complete audit trail

## 4. Lab environment

Each team uses a shared small physical ACI lab fabric for execution labs, a mocked APIC for failure paths, the DevNet ACI simulator for extra API practice, a ServiceNow personal developer instance and a GitHub organization. Hostnames are fictional (dc1-lab, apic1.example.corp).

### Components

For more info - https://genroor.com

| Component | Version or edition | Purpose | How to obtain | Notes |
|---|---|---|---|---|
| Physical ACI lab fabric | 3 APICs; 4 leaves (two vPC pairs) and 2 spines, Nexus 9000 in ACI mode; start on a 6.0(x) release and target the latest recommended 6.0(8) patch | Real APIC cluster and switch group upgrades | Training company lab or partner lab rental; Cisco lab hardware through a Cisco partner | Choose start and target releases with the Support Matrix before each cohort; 6.0(8e), 6.0(8f) and 6.0(8h) exist [18][19]. |
| Cisco DevNet ACI Simulator | Always-On (shared) and reservable (up to 5 days) | API practice and fixture recording without hardware | developer.cisco.com sandbox catalog, free | Shared state; not for upgrade execution [37]. |
| Mock APIC | Course-built Python service | Replays recorded responses for failure paths | Course repository | Recordings captured from the lab fabric. |
| Claude Code and Claude Agent SDK | Current release; Python SDK | Agent sessions and planning | Native installer or npm; pip install claude-agent-sdk | Requires a Claude Pro, Max, Team, Enterprise or Console account. |
| MCP Python SDK | mcp[cli], Python 3.11+ | Read-only APIC MCP server | pip or uv | Use the MCP Inspector for testing [30]. |
| Ansible with cisco.aci collection | Current cisco.aci from Ansible Galaxy | Policies and snapshots | ansible-galaxy collection install cisco.aci | Module docs in the CiscoDevNet/ansible-aci repository [22][23]. |
| ServiceNow PDI | Current release offered | Change requests and approval flow | ServiceNow Developer Site PDI guide, free | Request early; PDIs can be waitlisted or reclaimed when idle [35]. |
| GitHub organization with Actions | GitHub.com | Plan PRs and checks | Free organization per cohort | Branch protection needs a plan that supports it for private repos. |

## Fixtures and sample data

Gennoor Tech INNOVATE. BUILD. For more info - https://gennoor.com

| Fixture | Contents | Used in |
|---|---|---|
| Recorded APIC JSON | fabricNode, topSystem, firmwareRunning, firmwareCtrlrRunning, infraWiNode, faultInst, maintUpgJob responses | Fabric discovery through the APIC REST API; Executing the APIC cluster and switch group upgrades |
| Awkward fabric pack | Unpaired leaf, mixed models, degraded APIC, single-spine pod | Upgrade planning with structured outputs |
| Validator outputs | Pass run and bootflash failure run bundles | Pre-upgrade validation and explained failures |
| Failure recordings | Node not returning, 503 during conversion, missing node | Executing the APIC cluster and switch group upgrades |

### Lab safety rules

- Nothing touches production; lab fabric and PDI only.
- Bootflash fill only under instructor supervision on the named leaf, with cleanup verified.
- Never reload an APIC during an upgrade; stop rules from Cisco are posted in the room [15].
- Credentials only in environment variables or GitHub secrets.
- Instructor holds the CAB approval role.

## 5. Module-by-module content plan

### Day 1

#### Module 1: The Cisco ACI upgrade process and where upgrades go wrong

**Goal.** Learners can describe Cisco's recommended ACI upgrade sequence end to end and map every step to the APIC objects and checks it touches.

**Outline topics.**

- Cisco's recommended sequence: target release and release notes, compatibility matrix, pre-upgrade validation, APIC cluster, switch upgrade groups, post-checks
- Why vPC peers never share a group, why the cluster must be fully fit, and what bootflash space, faults and NTP drift do to an upgrade
- Firmware repository, maintenance policies, maintenance groups and the scheduler as APIC objects
- The teaching pattern: Claude plans and explains, deterministic code executes, a human approves, everything is logged

**Learning objectives.**

- Learners can list Cisco's upgrade workflow in order, from backup and image download through APIC cluster, switch groups and post-checks [2].
- Learners can explain the group design rules that keep one vPC peer, one APIC-connected leaf and one route-reflector spine up during every group [3].
- Learners can name the APIC objects that represent the firmware repository, controller and switch firmware targets, maintenance policies and groups [7][10].
- Learners can state the course pattern: Claude plans and explains, deterministic code executes, a person approves, everything is logged.

**Cisco's documented sequence**

- Walk the guide structure so learners know where each answer lives: overview, architecture, checklists, mixed-version operations, and GUI, REST and CLI procedures [1].
- The workflow order: AES-encrypted config export, disable App Center apps, download APIC and switch images to the APICs, pre-upgrade validation, CIMC/HUU update, APIC upgrade, validate again, wait for Fully Fit, then switches [2].
- A multistep upgrade repeats the whole cycle from the intermediate release; plan two windows, not one [2].
- Target release selection: use the Upgrade/Downgrade Support Matrix with from and to releases to see behavior changes and open bugs on the path [19].
- Read release notes for every release between current and target (Changes in Behavior) plus Open and Known Issues for the target, for both APIC and switches [12].
- CIMC is part of the plan: release notes say to install only the CIMC versions in the compatibility table, and point to advisories such as CSCwo74485 for specific APIC servers [9].

**Where ACI upgrades go wrong**

- Both vPC peers in one group: Cisco wants one leaf of each vPC pair running, and from 4.2(5) concurrency in a group is unlimited by default, so a careless group reboots both peers together [3].
- Unpaired leaves: the checklist warns that without vPC pairs the built-in protection against simultaneous leaf upgrades does not work [12].
- An APIC cluster that is not Fully Fit, uncleared faults, NTP drift between nodes and bootflash usage above the F1820/F1821 thresholds are all called out as blockers [12].
- Bootflash matters because the switch checks filesystem and bootflash space before extracting the image, and auto-cleanup above 50% can even delete the boot image [14][12].
- During a stalled upgrade the wrong reflex causes the damage: do not reload an APIC, do not decommission one, and do not flip the firmware target back [15].
- TAC experience: never reboot two or more APICs at the same time, and do nothing on APIC2 or APIC3 while APIC1 is still upgrading [20].

**The APIC object model for upgrades**

- Image repository: firmwareRepoP with a firmwareOSource child posted to /api/node/mo/uni/fabric.xml [7].
- Controller target and trigger: firmwareCtrlrFwP (version, ignoreCompat) and maintCtrlrMaintP (adminState up, adminSt triggered) under uni/controller [7].
- Switch policy and group from 4.0(1): maintMaintP carries version and runMode, maintMaintGrp holds fabricNodeBlk ranges and a maintRsMgrpp relation to the policy [10].
- Status classes for checking progress: firmwareCtrlrRunning, firmwareRunning and maintUpgJob [11].
- Scheduler note: from 5.1 the GUI no longer offers a scheduler for upgrades; the Ansible maintenance policy module still requires one, so the course keeps window enforcement in its own code [17][22].

**The teaching pattern for this course**

- Claude reads validator output, inventory and release notes, proposes plans and drafts reports; it never holds credentials that can write to the fabric.
- Python and Ansible create policies and trigger groups only after a GitHub pull request and a ServiceNow change are approved.
- Every prompt, tool call, APIC API call and approval lands in one audit log that the change board can read.
- The MCP specification itself says a human should be able to deny tool invocations and that tool use should be logged, which is the same rule applied to the fabric [29].

**Instructor demo.**

1. Open the Cisco upgrade guide and show the chapter list, then the workflow page, reading the order aloud [1][2].
2. Run the Support Matrix for the lab's current release to the course target release and show the behavior changes and open bugs it returns [19].
3. In the lab APIC GUI, open Admin > Firmware and show controllers, switches and images, then open the API Inspector and capture the calls the page makes.
4. Query /api/node/class/firmwareRunning.json and /api/node/class/maintUpgJob.json in a browser tab and relate the JSON to the GUI [11].
5. Draw the lab fabric (3 APICs, 4 leaves in two vPC pairs, 2 spines) and ask the room which two switches must never share a group [3].
6. Show the guidelines page section on what not to do when an upgrade stalls and put it on the wall as the course's stop rules [15].

**Hands-on lab.** Walk the lab fabric in the APIC GUI and API Inspector, map each Cisco process step to the APIC objects it touches, and write the workflow brief.

1. Log in to the lab APIC GUI and record the current APIC and switch versions from Admin > Firmware.
2. Open the API Inspector, refresh the Firmware pages and copy the class queries it shows into notes.md.

3. Run the same queries with curl after an aaaLogin and save the raw JSON under fixtures/ [8].
4. Fill the process table: for each Cisco workflow step, list the APIC object or class it reads or writes [2][7][10].
5. Mark the lab's vPC pairs, APIC-connected leaves and route-reflector spines on the topology sheet [3].
6. List the blockers the checklist names and note which ones the lab can show today (faults, NTP, bootflash, cluster health) [12].
7. Write the one-page ACI upgrade workflow brief: sequence, objects, stop rules and who approves what.
8. Peer-review another team's brief against the Cisco workflow page.

**Lab success criteria.**

- The brief lists every Cisco workflow step in the documented order, including validation before and after the APIC upgrade [2].
- Each step maps to at least one named APIC class or object.
- The topology sheet correctly identifies both vPC pairs and the spines that must stay up per group [3].
- The stop rules include no APIC reload and no target change during a stalled upgrade [15].

**Deliverable.** ACI upgrade workflow brief.

**Common pitfalls.**

- Treating the switch upgrade as one big group because concurrency is unlimited from 4.2(5) [3].
- Skipping the second validation run after the APIC upgrade [2].
- Choosing a target release from memory instead of the Support Matrix and release notes [19].
- Ignoring CIMC compatibility because 'it is only the server' [9].
- Confusing GUI scheduling with policy scheduling; from 5.1 the GUI has no scheduler option [17].

**Knowledge check.**

1. Why must the APIC cluster be Fully Fit before switch upgrades start? Answer: Cisco's workflow upgrades the APICs first and only moves to ACI-mode switches once all APICs are Fully Fit; a degraded cluster cannot safely orchestrate switch upgrades [2].
2. Which three things must stay up in every switch group? Answer: One leaf of each vPC pair, one leaf connected to each APIC, and at least one route-reflector spine and one IPN-connected spine per pod [3].
3. Which class tells you upgrade job status for both controllers and switches? Answer: maintUpgJob, queried at /api/node/class/maintUpgJob [11].
4. An upgrade stalls at 75% on APIC2. Name two things you must not do. Answer: Do not reload any APIC and do not switch the firmware target back to the original version; Cisco also says not to decommission a controller [15][20].

**Slides and assets to prepare.**

GenNoor Tech
TRAIN. INNOVATE. BUILD.
For more info: https://gennoor.com

- One-page diagram of Cisco's upgrade workflow with the two validation points highlighted.
- Lab topology poster: 3 APICs, leaf-101 to leaf-104 in two vPC pairs, spine-201 and spine-202.
- Table mapping each workflow step to APIC classes (firmwareRepoP, firmwareCtrlrFwP, maintCtrlrMaintP, maintMaintP, maintMaintGrp, maintUpgJob).
- 'Stop rules' card for stalled upgrades taken from the guidelines page.
- Workflow brief template (markdown).

**Sources for this module.** 1, 2, 3, 7, 8, 9, 10, 11, 12, 14, 15, 17, 19, 20, 22, 29

## Module 2: Claude Code, Agent SDK and MCP setup for the ACI workflow

**Goal.** Learners stand up a project where Claude can query the fabric through read-only MCP tools and return schema-shaped answers, with no path to write actions.

**Outline topics.**

- Installing Claude Code and the Claude Agent SDK in the project repository with a CLAUDE.md that encodes the upgrade rules
- Building an MCP server that exposes read-only APIC queries (nodes, firmware, faults, health) as tools
- Tool use with structured JSON outputs: defining the inventory and plan schemas Claude must return
- Separating read tools from write actions so the agent can observe but never trigger

**Learning objectives.**

- Learners can install Claude Code, create a project CLAUDE.md that encodes the upgrade rules, and confirm it loaded [24].
- Learners can build and register a stdio MCP server that exposes read-only APIC queries [25] [30].
- Learners can call the Agent SDK with an output schema and read structured_output from the result [26].
- Learners can configure permissions so the agent can observe the fabric but cannot run write commands [28].

**Project setup and CLAUDE.md**

- Install Claude Code with the native installer or npm and confirm with claude --version and claude doctor; a Pro, Max, Team, Enterprise or Console account is needed.
- Create ./CLAUDE.md (or ./.claude/CLAUDE.md); /init can draft one and /context shows whether it loaded [24].
- Encode the course rules there: APIC cluster first, vPC peers never in one group, never trigger a maintenance policy, always cite the query behind a claim.
- Keep the file under about 200 lines and import the long runbook with @docs/upgrade-rules.md [24].

Gennoor Tech TRAINING ACADEMY BUILD
For more info - https://gennoor.com

- Stress that CLAUDE.md is context, not enforcement; anything that must never happen is blocked by permissions or a PreToolUse hook [24][27].

**A read-only APIC MCP server**

- Install the Python SDK with pip install "mcp[cli]" (Python 3.11+) and register tools with @mcp.tool(); type hints and docstrings become the input schema [30].
- Expose only GET-backed tools: get_nodes (fabricNode), get_controller_firmware (firmwareCtrlrRunning), get_switch_firmware (firmwareRunning), get_faults (faultInst with severity filter), get_cluster_health (infraWiNode) [8][11].
- Each tool logs in with aaaLogin, refreshes with aaaRefresh before refreshTimeoutSeconds expires, and never accepts a URL or method from the model [8].
- Return structuredContent with an outputSchema so the client can validate tool results, and set isError for API failures instead of raising [29].
- Test with uv run mcp dev server.py in the MCP Inspector before connecting Claude [30].

**Registering the server with Claude Code**

- claude mcp add --transport stdio --env APIC_HOST=... --scope project apic-ro -- python mcp/apic_ro.py writes .mcp.json for the team [25].
- Use ${APIC_USER} and ${APIC_PASS} expansion in .mcp.json so credentials stay in the environment, not in the repository [25].
- Project-scoped servers need approval on first use; /mcp shows status and the tool count [25].
- Server-side rate limiting and input validation are the server's job under the MCP spec, not the model's [29].

**Structured outputs and permissions in the Agent SDK**

- Define Inventory and UpgradePlan as Pydantic models and pass output_format={'type': 'json_schema', 'schema': Model.model_json_schema()} [26].
- Read message.structured_output only when subtype is success; handle error_max_structured_output_retries as a failed run [26].
- The SDK validates against JSON Schema draft-07; keep schemas focused and make fields optional when data may be missing [26].
- Lock the agent down: allowed_tools=['mcp__apic-ro__*', 'Read'], disallowed_tools=['Bash', 'Write', 'Edit'], permission_mode='dontAsk' for headless runs [28].
- Remember allowed_tools does not limit bypassPermissions mode, and deny rules apply in every mode [28].

**Instructor demo.**

1. Show the finished repository layout: CLAUDE.md, .mcp.json, mcp/apic_ro.py, schemas/, runner/ and audit/.
2. Run /init in a fresh copy, then replace the generated text with the course rules and show /context listing the file [24].

For more info: https://gennoor.com

3. Start the MCP server in the Inspector and call get_switch_firmware against the lab APIC [30].
4. Register it with claude mcp add at project scope and show the resulting .mcp.json [25].
5. Ask Claude Code 'Which leaves are vPC peers and what are they running?' and point out that every fact came from a tool call.
6. Ask Claude to 'trigger the upgrade for leaf-101' and show that no tool exists and Bash is denied [28].

**Hands-on lab.** Stand up the MCP server against the lab APIC and have Claude Code answer fabric questions using only the exposed tools.

1. Clone the starter repository and install Claude Code; run claude doctor.
2. Write CLAUDE.md with the upgrade rules and import docs/upgrade-rules.md [24].
3. Implement the five read-only tools in mcp/apic_ro.py with aaaLogin and aaaRefresh handling [8] [30].
4. Add an outputSchema to each tool and return structuredContent [29].
5. Test every tool in the MCP Inspector against the lab APIC or the DevNet simulator [30][37].
6. Register the server at project scope with environment-variable credentials [25].
7. Add .claude/settings.json deny rules for Bash, Write and Edit outside the repo, and a hook that blocks any tool name containing 'trigger' [27].
8. Ask Claude Code five fabric questions and save the transcript to evidence/module2.md.

**Lab success criteria.**

- All five tools return schema-valid JSON from the lab APIC.
- No credential appears in the repository or the transcript.
- Claude Code answers each question with tool calls only and names the class it queried.
- An attempt to run a shell command or write a policy is refused by settings or hook [27][28].

**Deliverable.** Working Claude Code and MCP environment.

**Common pitfalls.**

- Letting a tool accept an arbitrary APIC URL, which turns a read-only server into a generic API proxy.
- Forgetting aaaRefresh so long sessions fail mid-demo after the session timeout [8].
- Putting the password in .mcp.json instead of using ${VAR} expansion [25].
- Relying on CLAUDE.md wording to stop write actions instead of permissions and hooks [24].
- Declaring a draft 2020-12 schema; the SDK validates with draft-07 [26].

**Knowledge check.**

1. Where should the rule 'never trigger a maintenance policy' be enforced? Answer: In permissions and hooks (and by not exposing any write tool); CLAUDE.md is context that Claude follows, not an enforced control [24][27].

2. What does an MCP tool return when the APIC call fails? Answer: A tool result with isError: true and a text explanation, rather than a protocol error [29].
3. Where does the validated plan appear in an Agent SDK run? Answer: In structured_output on the ResultMessage when subtype is success [26].
4. Why is permission_mode 'dontAsk' useful for a CI run? Answer: Any call that would prompt is denied, so only pre-approved tools run; there is no human to answer prompts in CI [28].

**Slides and assets to prepare.**

- Starter repository with CLAUDE.md template, empty MCP server and Pydantic schema stubs.
- Diagram: Claude Code and Agent SDK on one side, read-only MCP server in the middle, APIC on the other, write path drawn separately through runner code.
- Cheat sheet of claude mcp add flags and .mcp.json fields.
- settings.json example with deny rules and a PreToolUse hook.
- Recorded APIC JSON fixtures for offline practice.

**Sources for this module.** 8, 11, 24, 25, 26, 27, 28, 29, 30, 37

## Module 3: Fabric discovery through the APIC REST API

**Goal.** Learners build and test a Python discovery client that turns APIC class queries into a typed, validated fabric inventory.

**Outline topics.**

- POST to aaaLogin, token refresh via aaaRefresh, and class and filter queries against fabricNode, topSystem, infraWiNode and firmwareCtrlrRunning
- Typed inventory record: node id, role, model, running version, vPC peer, health score, cluster state
- Pydantic validation, unit tests with pytest, and rejected-row reporting for incomplete nodes

**Learning objectives.**

- Learners can authenticate to the APIC with aaaLogin and keep the session alive with aaaRefresh [8].
- Learners can write class queries with filters and subtree options for nodes, firmware and cluster state [8][11].
- Learners can define a Pydantic inventory record and reject incomplete nodes with a clear reason.
- Learners can test the client with pytest against recorded fixtures and review every Claude-written diff.

**Session handling**

- POST {"aaaUser": {"attributes": {"name": ..., "pwd": ...}}} to /api/aaaLogin.json; the token arrives as APIC-cookie and in the response [8].

Gennoor Tech INNOVATE. BUILD. For more info - https://gennoor.com

- Use the apic:domain\user form when the fabric has several login domains [8].
- Default session timeout is 600 seconds; call aaaRefresh before refreshTimeoutSeconds and store the new token each time [8].
- Wrap the session in a small class with retry on 401 and a hard timeout per request.

**The queries behind the inventory**

- fabricNode for id, role, model, serial and fabricSt; topSystem for address, uptime and state.
- firmwareCtrlrRunning for APIC versions and firmwareRunning for switch versions [11].
- infraWiNode for cluster membership and health as seen by each APIC.
- Use query-target-filter (for example eq(fabricNode.role,"leaf")), rsp-subtree-include=health and page/page-size for large fabrics [8].
- vPC pairing comes from the vPC protection group configuration; record the peer id for every leaf so the planner can split pairs [3].
- Capture each query string in the record so later explanations can cite it.

**Typed inventory and rejected rows**

- Inventory node fields: node_id, name, role, model, running_version, vpc_peer, health_score, apic_connected, pod, source_queries.
- Cluster fields: apic_id, version, health, and a cluster_fully_fit flag computed in code, never by the model.
- Pydantic validation turns a missing version or unknown role into a rejected row with a reason, not a silent default.
- Write inventory.json and rejected.json; a non-empty rejected file fails the pipeline step.

**Working with Claude Code on the client**

- Ask Claude Code to write the client in small steps: session, one query, model, then tests.
- Read every diff; reject code that hard-codes credentials or disables TLS verification.
- Generate pytest tests from recorded JSON fixtures, including a node without firmwareRunning data.
- Use the DevNet simulator for extra API practice; it is shared or time-limited, so record fixtures for repeatable tests [37].

**Instructor demo.**

1. curl the aaaLogin endpoint and show the token and refreshTimeoutSeconds in the response [8].
2. Run a fabricNode class query with a role filter, then the same with rsp-subtree-include=health [8].
3. Ask Claude Code to implement ApicSession with refresh, and review the diff line by line.
4. Add the Pydantic model and show validation rejecting a fixture with a missing version.
5. Run pytest and show the coverage of happy path, missing data and expired token.
6. Produce inventory.json for the lab fabric and open it next to the GUI.

GenNoor Tech
For more info: https://gennoor.com

**Hands-on lab.** Implement the discovery client and tests with Claude Code, inspect every diff, and produce inventory.json for the lab fabric.

1. Record fixtures from the lab APIC for fabricNode, topSystem, firmwareCtrlrRunning, firmwareRunning and infraWiNode [11].
2. Have Claude Code write the session class with aaaLogin, aaaRefresh and retry; review and commit [8].
3. Add query helpers with filters and paging [8].
4. Define InventoryNode and ClusterState Pydantic models.
5. Join the queries into one inventory and compute cluster_fully_fit in code.
6. Write rejected rows with reasons to rejected.json.
7. Write pytest tests for each query, the join and two broken fixtures.
8. Run against the lab and save inventory.json and the query log.

**Lab success criteria.**

- inventory.json lists all 3 APICs, 4 leaves and 2 spines with versions and vPC peers.
- A deliberately broken fixture produces a rejected row, not a crash or a default value.
- Tests pass offline using fixtures only.
- No credentials or disabled TLS checks in the committed code.

**Deliverable.** Fabric inventory client.

**Common pitfalls.**

- Treating an empty result as 'no faults' when the query itself failed.
- Letting the token expire during long paging loops [8].
- Using node names instead of node ids as keys; ids are what maintenance groups reference [10].
- Accepting Claude-generated code without reading it, especially around TLS and secrets.

**Knowledge check.**

1. What do you do before refreshTimeoutSeconds runs out? Answer: Call aaaRefresh and replace the stored token with the new one it returns [8].
2. Which options limit a class query to leaves and include health? Answer: query-target-filter=eq(fabricNode.role,"leaf") and rsp-subtree-include=health [8].
3. Why does the inventory store a vpc_peer field for every leaf? Answer: The planner must put vPC peers in different groups; Cisco requires one leaf of each pair to stay up [3].
4. A node has no running version in the response. What does the client do? Answer: It writes a rejected row with the reason and fails the step; it never guesses a version.

**Slides and assets to prepare.**

- APIC query reference card: classes, filters, subtree and paging options.
- Recorded JSON fixtures for the lab fabric, including two broken ones.
- Pydantic model skeleton for InventoryNode and ClusterState.

- Sample inventory.json and rejected.json.

**Sources for this module.** 3, 8, 10, 11, 37

## Module 4: Pre-upgrade validation and explained failures

**Goal.** Learners run Cisco's validators, parse their results into structured findings, and have Claude explain each failure with the query behind it and Cisco's remediation.

**Outline topics.**

- Running the APIC pre-upgrade validator and the Cisco aci-preupgrade-validation-script and parsing their results
- Checking faults, bootflash space, cluster health, NTP, CIMC and configuration backups against the target release
- Claude explains each failed check in plain language, cites the query that produced it and marks unknowns

**Learning objectives.**

- Learners can run the standalone pre-upgrade validation script and locate its result bundle [5][6].
- Learners can call the APIC pre-validation endpoints available from 5.1(1) and the built-in validator through an update group [4][10].
- Learners can check faults, bootflash, cluster health, NTP, CIMC and backups against the checklist [12][21].
- Learners can produce a validation report where Claude explains every FAIL in plain language and marks unknowns.

**Cisco's validation options**

- Built-in Pre-Upgrade Validator runs when an update group is configured; the App Center app is obsolete (App Center deprecated in 6.1.2); the standalone script runs from the APIC CLI [4].
- Run the script at least two weeks before the window so fixes have time to land [4].
- From 6.2(1) external rules are pulled into the Upgrade GUI through Intersight or a manual rules package [4].
- REST pre-validation from 5.1(1): GET /mqapi2/deployment.query.json?mode=validateCtrlrMaintP&targetVersion=..., and mode=validateSwitchMaintP or the async form with getSwitchPreValStatus polling [10].

**Running and parsing the script**

- Copy aci-preupgrade-validation-script.py to /data/techsupport, run python aci-preupgrade-validation-script.py, give admin credentials and pick a target version that is already on the APIC [5].
- The sample run reports 47 checks, from NTP status and switch /bootflash usage to VPC-paired leaf switches [6].

- Results land in /data/techsupport/preupgrade_validator_<timestamp>.tgz with a debug log, a JSON results file and a text dump [6].
- Parse the JSON results file, not the console text, into Finding records: check, status, detail, recommended action.
- Statuses the course handles explicitly: FAIL and MANUAL CHECK REQUIRED; anything not recognised becomes UNKNOWN [5].

**The checks that matter most**

- Faults: clear all faults before upgrading, except those expected from staging [12].
- Bootflash: F1820 at 80% and F1821 at 90%; the script compares free space with what the target needs, generally twice the image size [12][21].
- Cluster and switches: every APIC Fully Fit (acidiag cluster), all switches Active, standby APIC filesystems under 75% [12][21].
- NTP on APICs and switches with reachability; CIMC version per the release notes; an AES-encrypted config export [12][9].
- MD5 of the image checked on the file server, the uploaded copy, each APIC repository and each switch bootflash [12].

**Claude explains, code decides**

- Code decides pass or fail; Claude only writes the explanation, the likely cause and Cisco's documented remediation.
- Every explanation cites the finding id and the APIC query or script check that produced it.
- If the evidence does not show a cause, Claude must write 'unknown' and list what to collect next.
- Output is a ValidationReport schema through structured outputs so the explanation can be gated like the plan [26].

**Instructor demo.**

1. Copy the script to /data/techsupport on the lab APIC and run it against the course target release [5].
2. Extract the .tgz and open the JSON results file [6].
3. Show the bootflash and NTP checks in the docs and what each one compares [21].
4. Run the parser and show the Finding records.
5. Ask the agent for a ValidationReport and show one FAIL explained with its source query and remediation.
6. Show an UNKNOWN finding and how the report asks for more data rather than guessing.

**Hands-on lab.** Fill a leaf's bootflash on purpose on the lab fabric, run validation, and have Claude explain the failure and the Cisco-recommended remediation.

1. Record the current bootflash usage on leaf-104 with df -h and the eqptcapacityFSPartition class [12].

2. Under instructor supervision, copy filler files onto leaf-104 bootflash until usage passes the minor fault threshold [12].
3. Confirm fault F1820 or F1821 appears in faultInst [12].
4. Run the validation script and the REST switch pre-validation for the target release [5][10].
5. Parse the results into findings.json.
6. Ask the agent for a ValidationReport and save it.
7. Remove the filler files, re-run validation and record the PASS.
8. Attach both runs to the evidence folder.

**Lab success criteria.**

- The first run shows a bootflash finding for leaf-104 and the second run clears it.
- The report cites the check name and the query or fault behind it.
- The remediation text matches Cisco's guidance (free space by removing files, not by deleting images with rm on the APIC) [12].
- Any check without evidence is reported as unknown.

**Deliverable.** Validation report with explained findings.

**Common pitfalls.**

- Running the script with a target version that is not yet on the APIC [5].
- Deleting images on the APIC with rm instead of through Admin > Firmware [12].
- Trusting a summary line instead of parsing each check.
- Running validation once, weeks early, and not again after the APIC upgrade [2][4].
- Letting the model decide pass or fail.

**Knowledge check.**

1. How early does Cisco suggest running the standalone script? Answer: At least two weeks before the maintenance window [4].
2. What does the bootflash check compare? Answer: Each switch's free /bootflash space with the space the target release needs, generally twice the target image size [21].
3. Where are the script results saved? Answer: In a timestamped preupgrade_validator .tgz under /data/techsupport containing a debug log, JSON results and a text dump [6].
4. What should Claude write when it cannot tell why a check failed? Answer: Mark the cause unknown, cite the finding, and list what evidence to collect next.

**Slides and assets to prepare.**

- Table of the checklist items with the fault codes and thresholds.
- Sample script output and extracted JSON results.
- ValidationReport schema and a filled example.
- Safe bootflash fill and cleanup procedure for the lab, instructor only.

Gennoor Tech
INNOVATE BUILD
For more info - https://gennoor.com

**Sources for this module.** 2, 4, 5, 6, 9, 10, 12, 21, 26

## Day 2

### Module 5: Upgrade planning with structured outputs

**Goal.** Learners have Claude propose upgrade groups and windows as structured output, then prove that code rejects any plan that breaks Cisco's grouping rules.

**Outline topics.**

- Claude proposes upgrade groups from the inventory: APIC cluster first, vPC peers split, spines balanced, leaves balanced
- Mapping groups to maintenance windows and scheduler times in the fabric's local time zone
- JSON Schema validation of the plan and automatic rejection of plans that break a rule
- Testing the planner with awkward fabrics: odd vPC pairs, mixed models, a degraded APIC

**Learning objectives.**

- Learners can encode Cisco's group rules as code checks [3].
- Learners can generate an UpgradePlan as schema-validated structured output from inventory and validation results [26].
- Learners can map groups to maintenance windows in the fabric's local time zone.
- Learners can test the planner against awkward fabrics and show it rejects bad plans.

**Rules from Cisco, written as code**

- Order: APIC cluster first, switch groups only after the cluster is Fully Fit [2].
- At least two groups; vPC peers in different groups; one APIC-connected leaf per APIC up; at least one route-reflector spine and one IPN-connected spine per pod up [3].
- Separate redundant non-vPC leaves such as border leaves too [20].
- With graceful upgrade enforced, the APIC rejects upgrading the last spine in a pod; one-spine pods need graceful disabled [3].
- Run mode per group: pauseOnlyOnFailures is the APIC default and the course default [10][22].

**The plan schema**

- UpgradePlan: plan_id, fabric, target_version, intermediate_version, apic_step, groups[], windows[], risks[], unknowns[], source_inventory_hash.
- Group: name, node_ids, role_mix, run_mode, graceful, window_ref, rationale.
- Window: start and end in ISO 8601 with offset (for example +05:30 for IST), change_ref.
- Pass the Pydantic schema as output_format and read structured_output [26].
- Validation happens twice: JSON Schema shape in the SDK, then the rule checks in Python.

**Windows and time zones**

- Every window carries an explicit offset; the runner compares times in UTC.
- From 5.1 the GUI has no scheduler; the course runner decides when to trigger, inside the approved window [17].
- Allow time for pre-download before the install window; the 5.1+ workflow downloads first, then Begin Install [16].
- A multistep upgrade needs a window per hop [2].

**Testing the planner**

- Fixtures: the lab fabric, a fabric with an unpaired vPC leaf, mixed leaf models, a degraded APIC, a single-spine pod.
- Expected outcome per fixture written first, then the planner run.
- A degraded APIC means no plan, only a blocking finding [12].
- An unpaired leaf is flagged because vPC protection cannot work for it [12].

**Instructor demo.**

1. Show the rule checks in plan_rules.py next to the Cisco guideline text [3].
2. Run the planner for the lab fabric and print plan-v1.json.
3. Swap leaf-102 and leaf-103 by hand so both vPC peers sit in group A and run the gate; show the rejection message.
4. Run the degraded APIC fixture and show the planner returns a blocking finding instead of a plan.
5. Show a window with a missing offset failing schema validation [26].

**Hands-on lab.** Generate the plan for the lab fabric, feed it a fabric with an unpaired vPC leaf, and prove the schema gate rejects the bad plan.

1. Write the UpgradePlan Pydantic models.
2. Implement rule checks for vPC split, APIC-connected leaves, spines per pod and order [3].
3. Write the planner prompt with inventory.json and findings.json as inputs.
4. Run the planner with output_format and save plan.json [26].
5. Run the gate on plan.json.
6. Run the planner on the unpaired-leaf fixture and record the gate result.
7. Run the degraded APIC and single-spine fixtures.
8. Commit plans, gate results and tests.

**Lab success criteria.**

- The lab plan passes and puts leaf-101/leaf-102 and leaf-103/leaf-104 in different groups with one spine each.
- The unpaired-leaf fixture is flagged and a hand-edited bad plan is rejected with a named rule.
- The degraded APIC fixture produces no plan.
- All windows carry explicit offsets.

GenNoorTech
For more info - https://gennoor.com

**Deliverable.** Schema-validated upgrade plan.

**Common pitfalls.**

- Letting Claude's rationale text count as validation.
- Putting both spines of a two-spine pod in one group [3].
- Using local times without offsets.
- Forgetting that run mode and graceful are per policy, so groups with different settings need different policies [22].

**Knowledge check.**

1. What happens if both members of a vPC pair are in the same group? Answer: Cisco's rule is broken; the course gate rejects the plan. Cisco notes that in that case only one is upgraded at a time, but the course never relies on that [3].
2. Why does a one-spine pod matter when graceful upgrade is enforced? Answer: The APIC rejects upgrading the last spine in a pod under graceful upgrade, so the plan must disable graceful for that pod or the group fails [3].
3. Where does the validated plan come from in the SDK? Answer: structured_output on a successful ResultMessage; error_max_structured_output_retries means no valid plan [26].
4. What does the planner return for a degraded APIC? Answer: A blocking finding and no plan, because the cluster must be Fully Fit before switch upgrades [2].

**Slides and assets to prepare.**

- Rule table: Cisco text on the left, Python check on the right.
- UpgradePlan schema diagram.
- Awkward fabric fixture pack.
- Window timeline graphic for Sat and Sun IST windows.

**Sources for this module.** 2, 3, 10, 12, 16, 17, 20, 22, 26

## Module 6: Firmware staging, backups and the change request

**Goal.** Learners stage the target image, take and verify a configuration snapshot, and raise a ServiceNow change from the approved plan with one command.

**Outline topics.**

- Uploading firmware to the APIC repository with firmwareFirmware and firmwareOSource and verifying checksums
- Configuration export and snapshot with configExportP and configSnapshot before any change
- Creating the ServiceNow change request through the Table API with the plan, windows and risk text drafted by Claude
- Tracking approval state with Flow Designer and blocking execution until the change is approved

**Learning objectives.**

- Learners can register firmware sources and verify the image checksum at each point Cisco lists [7][12].
- Learners can take a configuration snapshot through configExportP and wait for it to complete [23].
- Learners can create a normal change through the ServiceNow API with plan, windows and risk text [31][33].
- Learners can read the change state and approval state and block execution until it is approved [32][34].

**Firmware repository**

- Register the image with firmwareRepoP and firmwareOSource (name, proto, url) at /api/node/mo/uni/fabric.xml [7].
- Images for APICs and switches are downloaded to the APICs; switches pull firmware from the APICs later [2].
- Check MD5 on the file server, after upload, in each APIC's repository and on each switch's bootflash [12].
- If the image is missing on an APIC, wait about five minutes, then remove and re-download through the GUI or API, never with rm [12].

**Backups and snapshots**

- Cisco asks for a configuration export with AES encryption before the upgrade [12].
- Record the encryption passphrase; TAC notes passwords will not import correctly without the key [20].
- Snapshots are created through configExportP with snapshot enabled; max_count defaults to 3 and rolls over [23].
- Pause and poll until the snapshot exists before comparing or relying on it [23].
- Store the snapshot name and export location in the evidence bundle.

**The ServiceNow change**

- Create a normal change through the Change Management API, or a change_request record through the Table API in a PDI [31][33].
- Fields: short_description, description (plan summary), justification, implementation plan, backout plan, risk and impact text drafted by Claude, start_date and end_date from the plan windows, cmdb_ci.
- Only one change is created per call; store the returned sys_id and number [31][33].
- Roles: change_manager, itil or admin for POST and PATCH [31].

**Approval tracking**

- Normal change states: New, Assess, Authorize, Scheduled, Implement, Review, Closed [32].

- The runner treats Scheduled (-2) or Implement (-1) with approval approved as the only go states [32].
- A Flow Designer flow with Ask for Approval and a Due Date notifies the CAB and stops waiting forever [34].
- Poll with GET and sysparm_fields to keep payloads small [33].

**Instructor demo.**

1. Post firmwareRepoP for the switch and APIC images and watch the download in Admin > Firmware > Images [7].
2. Compare MD5 values at each point and write them to evidence/md5.json [12].
3. Run the snapshot playbook with a pause, then list snapshots [23].
4. Run snow change create --plan plan.json against the PDI and show the record [31].
5. Show the flow sending an approval request and the change moving from Assess to Authorize [32][34].

**Hands-on lab.** Stage the target image, take the snapshot, and raise the lab ServiceNow change from the plan with one command.

1. Upload the target images to the lab file server and record MD5.
2. Register them on the APIC and wait for download [7].
3. Verify MD5 on the APIC repositories and record results [12].
4. Run the AES-encrypted export and the snapshot through the Ansible module with a pause [23].
5. Ask Claude to draft the change text from plan.json and review it.
6. Run the change create command against the PDI [31][33].
7. Build or import the approval flow with a Due Date [34].
8. Poll the change and confirm the runner reports 'blocked: not approved'.

**Lab success criteria.**

- Images present on all APICs with matching MD5.
- A completed snapshot whose name is recorded in evidence.
- One change record created from one command, with windows matching the plan.
- The runner refuses to proceed while the change is not in an approved state.

**Deliverable.** Staged firmware, snapshot and change request.

**Common pitfalls.**

- Comparing snapshots before they finish [23].
- Losing the export passphrase [20].
- Creating duplicate change records on retry; check for an existing change by plan_id first.
- Treating Assess as approved [32].

**Knowledge check.**

1. Which class creates a configuration snapshot? Answer: configExportP; configSnapshot is used to remove snapshots [23].
2. Name the four points where Cisco says to check image MD5. Answer: On the file server or desktop, the uploaded APIC copy, each APIC's firmware repository, and each switch's bootflash [12].
3. Which change states let the runner proceed? Answer: Only an approved change in Scheduled or Implement; New, Assess and Authorize block [32].
4. Why does the approval flow have a Due Date? Answer: So the flow does not wait forever for an approval [34].

**Slides and assets to prepare.**

- Firmware staging checklist with MD5 table.
- Ansible snapshot playbook.
- Change request field mapping from plan.json.
- ServiceNow PDI setup guide with sample flow export.

**Sources for this module.** 2, 7, 12, 20, 23, 31, 32, 33, 34, 35

## Module 7: Guardrails, approvals and audit for fabric upgrades

**Goal.** Learners build the gates that keep the agent read-only, require approved pull requests and changes, enforce windows and health, and record everything in an audit log.

**Outline topics.**

- The agent reads; code writes. Maintenance and firmware policies are created only by deterministic Python or Ansible cisco.aci modules
- Plan review and approval in a GitHub pull request with GitHub Actions running the validator and schema checks
- Window enforcement: no trigger outside the approved schedule, no next group while the last one is unhealthy
- Audit log of prompts, tool calls, API calls, approvals and outcomes in a format the change board can read

**Learning objectives.**

- Learners can show that only deterministic Python or Ansible creates maintenance and firmware policies [10][22].
- Learners can configure a pull request gate with required reviews and status checks [36].
- Learners can enforce windows and health so no group starts outside its window or after an unhealthy group [11].
- Learners can produce an audit log a change board can read.

**The agent reads; code writes**

- No write tool on the MCP server; Bash, Write and Edit denied for the agent session [28].
- A PreToolUse hook denies any tool call whose name or arguments mention maintMaintP, adminSt or triggered [27].
- Policies are created by runner/apply.py or Ansible cisco.aci modules with admin_state untriggered [22].
- The trigger is a separate step that checks change state, window and health before posting adminSt=triggered [10].

**Pull request and change gates**

- plan.json lives in a branch; a PR to main requires one approval from netops-lead and dismisses stale approvals on new commits [36].
- Required status checks: schema validation, rule checks and a fresh validator result [36].
- Apply to admins too so nobody bypasses the gate [36].
- The apply job also checks the ServiceNow change state and refuses if not approved [32].

**Window and health enforcement**

- The runner refuses to trigger if now is outside the window from the approved change.
- No next group until every node in the last group runs the target version and the cluster is Fully Fit [11][2].
- If validation is incomplete or stale, the next step is blocked.
- Do not change the switch firmware group mid-upgrade; Cisco says the upgrade will not complete [15].

**Audit log**

- Events: prompt, model output hash, tool call, APIC request and response code, approval, gate decision, operator.
- Append-only JSON lines with a run id, signed off by hash chaining each line.
- The MCP spec expects clients to log tool usage for audit [29].
- Render a change-board summary from the log: who approved, what ran, what was refused.

**Instructor demo.**

1. Open PR #87 with plan.json and show checks running.
2. Run make apply before approval and show the refusal and the audit entry.
3. Show the hook blocking the agent from calling a fake trigger tool [27].
4. Approve the PR and the change in the PDI, re-run apply, and show policies created with adminSt untriggered [22].
5. Try to trigger outside the window and show the refusal.

**Hands-on lab.** Open the plan as a pull request, try to apply it before approval and watch the gate refuse, then approve and apply to the lab.

1. Configure branch protection on main with one required review and required checks [36].
2. Write the GitHub Actions workflow that runs schema, rules and validator freshness checks.
3. Add the change-state check to apply.py [32].
4. Add the hook and deny rules for the agent session [27][28].
5. Open the plan PR and attempt apply before approval.
6. Approve PR and change; apply and confirm policies exist untriggered.
7. Attempt a trigger outside the window.
8. Export the audit log and the change-board summary.

**Lab success criteria.**

- Apply before approval is refused and logged.
- After approval, maintenance policy and group exist with adminSt untriggered.
- Trigger outside the window is refused.
- The audit summary lists every gate decision with time and actor.

**Deliverable.** Approval gate and audit log.

**Common pitfalls.**

- Admins bypassing branch protection because 'include administrators' is off [36].
- Creating the policy with admin_state triggered by mistake [22].
- Checking the window in local time without an offset.
- Logging secrets in the audit file.

**Knowledge check.**

1. Which attribute actually starts a switch group upgrade? Answer: adminSt="triggered" on the maintenance policy (maintMaintP) [10].
2. What does dismissing stale approvals protect against? Answer: A reviewer approving one plan and a different plan being merged after new commits [36].
3. Why is a PreToolUse hook used as well as missing tools? Answer: Hooks run first and can deny a call even in permissive modes, giving defence in depth [27][28].
4. What must be true before the next group starts? Answer: All nodes in the last group report the target version, maintUpgJob shows completion, the cluster is Fully Fit and the window is still open [11].

**Slides and assets to prepare.**

- Gate diagram: PR, checks, change, window, health, trigger.
- GitHub Actions workflow file.
- settings.json and hook script.
- Audit log schema and change-board summary template.

**Sources for this module.** 2, 10, 11, 15, 22, 27, 28, 29, 32, 36

Genmoor Tech
For more info - https://genmoor.com

## Day 3

### Module 8: Executing the APIC cluster and switch group upgrades

**Goal.** Learners run a deterministic runner that upgrades the APIC cluster, then each switch group inside its window, and pauses safely when a node does not return.

**Outline topics.**

- Upgrading the APIC cluster first and polling infraWiNode and firmwareCtrlrFwStatusCont until the cluster is fully fit
- Creating maintenance policies and groups per the plan and triggering each group only inside its window
- Polling firmwareRunning and maintUpgJob per node, with timeouts, retries and a pause when a node does not return
- Claude drafts the status update for the on-call engineer when the runner pauses

**Learning objectives.**

- Learners can drive the APIC cluster upgrade by REST and wait for Fully Fit [7][13].
- Learners can create and trigger switch maintenance policies and groups per the plan [10][22].
- Learners can poll firmwareRunning and maintUpgJob with timeouts and pause on failure [11].
- Learners can have Claude draft an on-call status update from runner state.

**APIC cluster upgrade**

- Set firmwareCtrlrFwP version, then post maintCtrlrMaintP adminSt=triggered [7].
- APICs install in parallel but convert and reload one at a time; the API is unavailable on the APIC that is converting [13].
- Poll firmwareCtrlrRunning and the cluster health from a different APIC than the one reloading [11].
- Before 6.0(6) done means all APICs online and Fully-Fit; from 6.0(6) wait for Post Upgrade Pending to become Completed [13].
- Never reload an APIC during conversion; if stuck, follow Cisco's stop rules and TAC steps [13] [15][20].

**Switch groups**

- From 4.0(1): maintMaintP with version and runMode, maintMaintGrp with fabricNodeBlk and maintRsMgrpp [10].
- Pre-download first; install only when all nodes are ready [16].
- Trigger with adminSt=triggered inside the window [10].
- Each switch checks bootflash, extracts, updates grub, may upgrade BIOS/EPLD and reloads, sometimes twice for SSD firmware [14].

**Polling, timeouts and pause**

- Poll maintUpgJob and firmwareRunning per node every 30 seconds with a per-node timeout from the plan [11].
- If a node misses its timeout, pause: do not trigger the next group, keep polling, alert on-call.
- Run mode pauseOnlyOnFailures matches this behavior on the APIC side [10][22].
- Retries are for reads only; triggers are never retried automatically.

**Recorded failure paths**

- A mocked APIC replays recorded responses: node stuck at a status, node missing from firmwareRunning, 503 during conversion.
- The runner is tested against each recording in pytest.
- Claude drafts the status update from runner state: what ran, what is stuck, what was not done, next decision owner.
- The draft is posted only after a person reads it; the runner never pages anyone with model text alone.

**Instructor demo.**

1. Show runner state machine: cluster, wait fully fit, group A, verify, group B.
2. Run the cluster step on the lab and narrate the serial reloads [13].
3. Trigger group A in the window and watch maintUpgJob [11].
4. Replay the node-not-returning recording and show the pause and the drafted status update.
5. Show the audit lines written for each step.

**Hands-on lab.** Run the APIC cluster upgrade and group A on the physical lab fabric, then replay a recorded node-not-returning response from a mocked APIC and watch the runner pause.

1. Confirm change approved, window open and fresh validation PASS.
2. Run the cluster step and wait for Fully Fit or Completed [13].
3. Re-run validation after the APIC upgrade [2].
4. Create group A policies untriggered, review, then trigger in window [10].
5. Poll until group A completes and record per-node durations [11].
6. Start the mock APIC with the node-not-returning recording.
7. Run the runner against the mock and confirm it pauses and does not trigger group B.
8. Save the drafted status update and audit log.

**Lab success criteria.**

- APIC cluster on target and Fully Fit before any switch moves.
- Group A on target with no trigger outside the window.
- Against the mock, the runner pauses and never triggers group B.
- The status update states facts from runner state only.

GenNoorTech
For more info - https://gennoor.com

**Deliverable.** Deterministic upgrade runner.

**Common pitfalls.**

- Polling the APIC that is converting and treating its silence as failure [13].
- Retrying a trigger after a timeout.
- Changing the firmware group mid-upgrade [15].
- Skipping the second validation run [2].

**Knowledge check.**

1. In what order do APICs convert and reload? Answer: One at a time; the next starts only after the previous one is back online [13].
2. Which three classes do you poll during switch upgrades? Answer: maintUpgJob for job status, firmwareRunning for switch versions and firmwareCtrlrRunning for controllers [11].
3. What does the runner do if a node misses its timeout? Answer: Pause, keep reading, alert on-call and wait for a person; it never starts the next group or re-triggers.
4. What status means a 6.0(6)+ APIC upgrade is finished? Answer: Completed, after Post Upgrade Pending [13].

**Slides and assets to prepare.**

- Runner state machine diagram.
- Mock APIC with recorded responses.
- Per-node timing sheet.
- Status update template.

**Sources for this module.** 2, 7, 10, 11, 13, 14, 15, 16, 20, 22

## Module 9: Post-upgrade verification, reporting and rollback decisions

**Goal.** Learners compare pre and post state in code, have Claude write the report and recommendation, and make a documented go, hold or roll back decision.

**Outline topics.**

- Pre and post snapshots of versions, faults, endpoint counts, routes, vPC and interface state compared by code
- Claude turns the diff into a readable report with a go, hold or roll back recommendation
- Rollback and downgrade options as Cisco documents them, and when to stop the next group
- Lessons captured into the runbook for the next fabric

**Learning objectives.**

- Learners can capture pre and post snapshots of versions, faults, endpoints, routes, vPC and interfaces and diff them in code [8][11].

Gennoor Tech
TRAINING INNOVATION BUILD
For more info - https://gennoor.com

- Learners can produce a readable report with a go, hold or roll back recommendation signed by a person.
- Learners can explain Cisco's downgrade constraints and when to stop the next group [12][15].
- Learners can update the runbook with lessons for the next fabric.

**What to compare**

- Versions: firmwareRunning and firmwareCtrlrRunning against the target [11].
- Faults: faultInst counts by severity, ordered by severity [8].
- Endpoints, routes, vPC and interface state from class queries taken before group A and after each group.
- The comparison is code: counts, sets and thresholds, written to diff.json.

**Report and recommendation**

- Claude receives diff.json and writes the report with sections: what changed, what did not, concerns, recommendation.
- Recommendation values are fixed: GO, HOLD, ROLLBACK; the schema rejects anything else [26].
- Any new critical fault or vPC down means HOLD by rule, whatever the text says.
- A person signs the decision and the signature goes in the audit log.

**Rollback and downgrade**

- Downgrade has its own checklist: features not supported on older releases must be removed first [12].
- Downgrading APICs from 6.2 or newer to before 6.2(1) is not supported [15].
- The old switch image sits in the recovery partition after upgrade [14].
- Config rollback from a snapshot is a separate, approved change [23].
- Stopping the next group is the usual first move; the guide has a chapter on which operations are allowed while switches run mixed versions, so read it before holding for long [1].

**Runbook capture**

- Record per-node durations, surprises and fixes.
- Update CLAUDE.md rules if the planner needs a new constraint [24].
- Add new fixtures for failure modes seen.
- Note which validator checks were MANUAL CHECK REQUIRED and how the team resolved them [5].

**Instructor demo.**

1. Run collect --phase pre and --phase post-A and show diff.json.
2. Generate the report and show the fixed recommendation field.
3. Introduce a synthetic new critical fault in the post fixture and show HOLD forced by rule.

GenMor Tech
For more info - https://genmor.com

4. Walk the downgrade checklist and the 6.2 limit [12][15].

**Hands-on lab.** Compare pre and post state for group A, generate the report, and make the go or hold call for group B as a team.

1. Load pre-upgrade snapshot data.
2. Collect post-group-A data.
3. Run the diff and review it.
4. Generate the report through structured output [26].
5. Discuss as a team and record GO or HOLD with signature.
6. If GO, schedule group B in the next window; if HOLD, write the hold note.
7. Add three runbook entries.

**Lab success criteria.**

- diff.json covers versions, faults, endpoints, routes, vPC and interfaces.
- The report recommendation matches the rules.
- The decision is signed and logged.
- Runbook updated.

**Deliverable.** Post-upgrade report and runbook.

**Common pitfalls.**

- Comparing against a stale pre snapshot.
- Letting report prose override a rule-based HOLD.
- Planning a downgrade without the downgrade checklist [12].
- Expecting endpoint counts to match exactly during learning after reboot; define tolerances.

**Knowledge check.**

1. Who decides go or hold? Answer: A person, using the code diff and Claude's report; rules can force HOLD.
2. Can you downgrade APICs from 6.2 to 6.1? Answer: No; Cisco says downgrading from 6.2 or newer to a release older than 6.2(1) is not supported [15].
3. Where does the previous switch image go after upgrade? Answer: To the recovery partition while the primary grub partition is updated to the target [14].
4. What is the first safe move when group A looks wrong? Answer: Hold group B and investigate; rollback is a separate approved change.

**Slides and assets to prepare.**

- diff.json example and report template.
- Decision record template with signature.
- Downgrade checklist summary.
- Runbook template.

Gennoor Tech TRAIN INNOVATE BUILD for more info - https://gennoor.com

**Sources for this module.** 1, 8, 11, 12, 14, 15, 23, 24, 26

## Module 10: Capstone: end-to-end ACI fabric upgrade with evidence bundle

**Goal.** Teams run the full workflow from inventory to upgraded fabric and hand over a complete evidence bundle and limitations statement.

**Outline topics.**

- Inventory in, upgraded fabric out: discovery, validation, plan, change, approval, execution, verification
- Evidence bundle with validator output, plan, approvals, API log and post-check report
- Demo and limitations statement

**Learning objectives.**

- Learners can run discovery, validation, plan, change, approval, execution and verification in sequence.
- Learners can assemble an evidence bundle a change board accepts.
- Learners can state the limitations of their workflow honestly.

**Capstone flow**

- Inventory, validation, plan, PR, change, approval, staging, snapshot, cluster, group A, verify, group B, verify [2].
- Each stage has an entry check and an evidence file.
- Failures are expected; how the team stops matters more than finishing.
- The instructor plays the CAB and can reject a window or a plan at any point.

**Evidence bundle**

- Validator output .tgz and parsed findings [6].
- plan.json, PR link, change number and approval record.
- Snapshot name, MD5 table, audit log, diff.json and report.
- Zipped as run-<date>.zip with a manifest.
- The manifest lists a SHA-256 for every file so the bundle can be checked later.

**Limitations statement**

- What the workflow does not cover: Multi-Pod, remote leaf, Multi-Site, catalog-only upgrades.
- What a person must still do: release selection review, CAB, judgement calls.
- Known gaps in fixtures and tests.
- Which Cisco checks were run by hand rather than by the pipeline.

**Instructor demo.**

1. Show a model evidence bundle and its manifest.

Gennoor Tech TRAINING INNOVATION BUILD For more info - https://gennoor.com

2. Walk the scoring rubric.
3. Show a short sample demo from a previous cohort.
4. Reset the lab fabric to the start release and show the team the starting inventory they must reproduce.

**Hands-on lab.** Capstone build and demo.

1. Reset the lab fabric to the start release.
2. Run discovery and validation.
3. Plan and open the PR; raise the change.
4. Get approval from the instructor acting as CAB.
5. Stage, snapshot, upgrade cluster and groups.
6. Verify and decide after each group.
7. Assemble the bundle and the limitations statement.
8. Demo in 15 minutes.

**Lab success criteria.**

- Fabric on target with no unapproved trigger.
- Bundle complete against the manifest.
- Every stop or failure is logged and explained.
- Limitations statement is specific.

**Deliverable.** End-to-end upgrade workflow and evidence bundle.

**Common pitfalls.**

- Skipping evidence steps to finish faster.
- Hand-editing plan.json after approval.
- Vague limitations.

**Knowledge check.**

1. What must be in the evidence bundle? Answer: Validator output, plan, approvals, API and audit log, snapshot and MD5 records, diff and report.
2. What happens if the plan changes after approval? Answer: Stale approvals are dismissed and the PR and change must be approved again [36].
3. Name one scope limit to state. Answer: For example Multi-Pod, remote leaf or Multi-Site fabrics are not covered.

**Slides and assets to prepare.**

- Capstone brief and rubric.
- Evidence bundle manifest.
- Demo script template.

**Sources for this module.** 2, 6, 36

Gennoor Tech TRAIN. INNOVATE. BUILD. For more info - https://gennoor.com

## 6. Capstone plan

A repeatable workflow that discovers a Cisco ACI fabric, runs the pre-upgrade validator, has Claude propose a schema-validated upgrade plan, raises and tracks the ServiceNow change, executes the APIC cluster and switch groups through deterministic code only after human approval, and ships a post-upgrade report with a full audit trail.

Teams take the lab fabric from the start release to the target release with the full workflow and ship an evidence bundle and limitations statement.

### Milestones

1. Inventory and validation PASS
2. Plan PR approved and change approved
3. Images staged and snapshot taken
4. APIC cluster upgraded and Fully Fit
5. Group A upgraded and verified
6. Group B decision and run
7. Evidence bundle and demo

### Evaluation notes

- No trigger without approval and window: pass/fail.
- Evidence completeness against the manifest.
- Quality of stops and explanations when something fails.
- Clarity of the report and limitations statement.

### Assessment weights

| Assessment area | Weight | Evidence |
|---|---|---|
| Working functionality | 25% | Demonstrable end-to-end workflow in a repeatable lab |
| Security and safety controls | 20% | Threat model, authorization boundaries, approval gates, safe failure |
| Testing and evaluation | 20% | Automated tests, golden dataset, measured quality and known limitations |
| Engineering quality | 15% | Clear interfaces, error handling, reproducible setup, maintainable code |
| Evidence and explainability | 10% | Traceable findings, sources, assumptions, logs |
| Documentation and presentation | 10% | Architecture diagram, README, runbook, demo, lessons learned |

Gennoor Tech
INNOVATE. BUILD.
For more info - https://gennoor.com

## 7. Promotional video: research behind the script

**Opening scenario.** The opening scenario is realistic: from APIC 4.2(5) a group upgrades all its switches at once by default, so a group that contains both members of a vPC pair can take both down together; Cisco's rules require one leaf of each vPC pair to stay up and recommend separate groups for separate schedules [3]. 'Two hundred switches' and 'one weekend window' are a teaching scenario, not a sourced statistic.

Gennoor Tech
TRAIN · INNOVATE · BUILD
For more info - https://gennoor.com

| Claim or on-screen text | Basis |
|---|---|
| 'A cluster that must be fully fit before anything moves' | Cisco's workflow waits for all APICs to be Fully Fit before switch upgrades [2][13]. |
| 'Why a vPC pair must never sit in the same group' | Supported by Cisco guidelines and TAC best practices [3][20]. |
| Terminal: apic1-3 on 6.0(5h), validate --target 6.0(8e) | 6.0(8e) is a real release, available October 28, 2024, with later 6.0(8f) and 6.0(8h) builds [18]; 6.0(5h) was not checked against release notes and is a lab fixture value. Instructors should use the current recommended patch. |
| '1 FAIL (bootflash < 3 GB on leaf-104)' | Illustrative. The script compares free /bootflash space with what the target needs, generally twice the image size, not a fixed 3 GB [21]. Suggest the on-screen text read 'insufficient /bootflash space on leaf-104'. |
| 'see query faultInst' | Bootflash usage raises faults F1820/F1821 and faults are queried through faultInst [12][8]. |
| Group A leaf-101, leaf-103 + spine-201; group B leaf-102, leaf-104 + spine-202 | Matches Cisco's ODD/EVEN style split with one vPC peer and one spine per group [3]. |
| 'window: Sat 02:00-06:00 IST' | Teaching scenario; from 5.1 the GUI has no scheduler, so the course enforces windows in its runner [17]. |
| 'CHG0041233 created state: assess' | Assess is a real normal-change state [32]; the number is fictional, and a new change normally starts in New before moving to Assess. |
| 'maint_policy group-A ... adminState: untriggered' | Maintenance policies default to untriggered [22]; the switch policy attribute is adminSt (adminState appears on the controller policy) [7] [10]. Suggest 'adminSt: untriggered' on screen. |
| 'apic cluster 6.0(8e) fully fit at 02:41', 'longest reboot 11m', 'endpoints 1,204 → 1,204', '214 events' | Illustrative run output, not facts. From 6.0(6) the APIC status reads Post Upgrade Pending then Completed [13]. |
| '10 hands-on modules', '3 days' | Matches the course outline in content.py. |
| '6 physical switches upgraded in the lab' | Course commitment: 4 leaves and 2 spines in the lab fabric. Requires the physical lab; the DevNet simulator cannot replace it [37]. |
| '0 upgrades without approval' | Design commitment enforced by PR, change, window and permission gates [27][28][36]. |
| 'The Cisco pre-upgrade validator and the open-source validation script' | Both are Cisco validation options [4][5]. |

Gennoor Tech
For more info: https://gennoor.com

| | |
|---|---|
| 'Rollback and downgrade options as Cisco documents them' | Covered by the downgrade checklist and guidelines [12][15]. |

## References


**[1]** Cisco APIC Installation and ACI Upgrade and Downgrade Guide. Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-installation-aci-upgrade-downgrade/Cisco-APIC-Installation-ACI-Upgrade-Downgrade-Guide.html
Supports: Course foundation; The Cisco ACI upgrade process and where upgrades go wrong. The guide is organized into an ACI Firmware Upgrade Overview, an Upgrade/Downgrade Architecture chapter, Pre-Upgrade/Downgrade Checklists, mixed-version operation guidance, and per-interface procedures (GUI by release including 5.1 or later and 6.2 or later, REST API, CLI).

**[2]** Upgrade or downgrade Cisco ACI fabrics (workflow). Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-installation-aci-upgrade-downgrade/Cisco-APIC-Installation-ACI-Upgrade-Downgrade-Guide/g-aci-firmware-upgrade-overview/workflow-to-upgrade-downgrade-apic.html
Supports: The Cisco ACI upgrade process; Executing the APIC cluster and switch group upgrades. Cisco's workflow: back up the configuration with AES encryption, disable App Center apps, download APIC and switch images to the APICs, run pre-upgrade validation, update CIMC/BIOS/HUU components, upgrade the APICs, validate again, wait until all APICs are Fully Fit, then upgrade the ACI-mode switches; a multistep upgrade repeats the cycle from the intermediate release.

**[3]** Guidelines for ACI switch upgrades and downgrades. Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-installation-aci-upgrade-downgrade/Cisco-APIC-Installation-ACI-Upgrade-Downgrade-Guide/g-aci-firmware-upgrade-overview/guidelines-for-aci-switch-upgrades.html
Supports: Upgrade planning with structured outputs; The Cisco ACI upgrade process. Divide leaf and spine switches into at least two update groups (ODD/EVEN example) so one leaf of each vPC pair, one leaf connected to each APIC, and at least one route-reflector spine and one IPN-connected spine per pod stay up; from APIC 4.2(5) switches in a group upgrade simultaneously with unlimited default concurrency, so Cisco recommends separate groups for separate schedules. Graceful upgrade uses GIR before reboot, the APIC rejects upgrading the last spine in a pod when graceful upgrade is enforced (from 4.1(1)), and switches already in GIR maintenance mode are not supported for firmware upgrades.

**[4]** Pre-upgrade validation tools. Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-installation-aci-upgrade-downgrade/Cisco-APIC-Installation-ACI-Upgrade-Downgrade-Guide/g-pre-upgrade-checklists/check-configurations-and-conditions-that-may-cause-an-upgrade-or-downgrade-failure.html
Supports: Pre-upgrade validation and explained failures. Three validation options exist: the built-in Pre-Upgrade Validator that runs automatically when an update group is configured, the App Center app (obsolete, App Center deprecated in 6.1.2), and the standalone script run from the APIC CLI; Cisco advises running the script at least two weeks before the maintenance window, and from APIC 6.2(1) external rules are pulled into the Upgrade GUI via Intersight or a manually uploaded rules package.

**[5]** GitHub - datacenter/ACI-Pre-Upgrade-Validation-Script. Cisco (datacenter GitHub org). https://github.com/datacenter/ACI-Pre-Upgrade-Validation-Script
Supports: Pre-upgrade validation and explained failures. Copy aci-preupgrade-validation-script.py to /data/techsupport on an APIC, run it with python, enter admin-level credentials and choose a target version that is already on the APIC; checks flagged FAIL or MANUAL CHECK REQUIRED come with recommended actions and the run ends with a count per status.

**[6]** Introduction - ACI Pre-Upgrade Validation Script documentation. Cisco (datacenter GitHub org). https://datacenter.github.io/ACI-Pre-Upgrade-Validation-Script/

Supports: Pre-upgrade validation and explained failures. The example run reports TOTAL: 47 checks, including NTP Status, Switch Node /bootflash usage and VPC-paired Leaf switches (shown as MANUAL CHECK REQUIRED), and saves output plus debug data to /data/techsupport/preupgrade_validator_<timestamp>.tgz, which contains a debug log, a JSON results file and a text dump.

**[7]** Upgrade or downgrade the Cisco APIC software using the REST API. Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-installation-aci-upgrade-downgrade/Cisco-APIC-Installation-ACI-Upgrade-Downgrade-Guide/g-upgrading-the-software-using-the-rest-api/upgrading-apic-controller-software-using-api.html
Supports: Firmware staging, backups and the change request; Executing the APIC cluster and switch group upgrades. A controller upgrade via REST is three POSTs: register the image source with firmwareRepoP/firmwareOSource (name, proto, url) at /api/node/mo/uni/fabric.xml, set the target version with firmwareCtrlrFwP (version, ignoreCompat) at /api/node/mo/uni/controller.xml, then trigger with maintCtrlrMaintP adminState="up" adminSt="triggered" at the same controller URL.

**[8]** Cisco APIC REST API Configuration Guide, Release 4.2(x) and Later - Using the REST API. Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-rest-api-configuration-guide/cisco-apic-rest-api-configuration-guide-42x-and-later/m_using_the_rest_api.html
Supports: Fabric discovery through the APIC REST API. Clients POST an aaaUser name/pwd to /api/aaaLogin.json (prefixing apic:domain\ when several login domains exist), get a token in the APIC-cookie and the refreshTimeoutSeconds attribute (default session timeout 600 seconds), and call aaaRefresh before expiry, storing the new token each time. Class queries use /api/node/class/<class>.json and object queries /api/node/mo/<dn>.json, with options query-target (self, children, subtree), target-subtree-class, query-target-filter such as eq(...), rsp-subtree-include=faults,health, order-by and page/page-size (example order-by=faultInst.severity|desc&page=0&page-size=15).

**[9]** Cisco APIC Release Notes, Release 6.1(4). Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/6x/release-notes/cisco-apic-release-notes-614.html
Supports: The Cisco ACI upgrade process (release notes and compatibility). Release 6.1(4h) became available on August 13, 2025; the notes say to install only the CIMC versions in the compatibility table (other firmware can stop an APIC booting) and point to software advisory CSCwo74485 before upgrading listed UCS C225 M6 (APIC-L4/M4) CIMC versions.

**[10]** Upgrade or downgrade switch software using the REST API. Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-installation-aci-upgrade-downgrade/Cisco-APIC-Installation-ACI-Upgrade-Downgrade-Guide/g-upgrading-the-software-using-the-rest-api/upgrading-switches-software-using-api.html
Supports: Executing the APIC cluster and switch group upgrades; Guardrails, approvals and audit for fabric upgrades. Switch upgrades via REST: add the image with firmwareRepoP/firmwareOSource at /api/node/mo/uni/fabric.xml; from 4.0(1) create a maintMaintP with version and runMode="pauseOnlyOnFailures" and a maintMaintGrp containing fabricNodeBlk (from_/to_) entries and a maintRsMgrpp relation to the policy; trigger by posting maintMaintP with adminSt="triggered". From 5.1(1) the guide adds pre-validation calls such as /mqapi2/deployment.query.json?mode=validateCtrlrMaintP&targetVersion=... and mode=validateSwitchMaintP / validateSwitchMaintPAsync with getSwitchPreValStatus polling.

**[11]** Verify the firmware version and the upgrade status using the API. Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-installation-aci-upgrade-downgrade/Cisco-APIC-Installation-ACI-Upgrade-Downgrade-Guide/g-upgrading-the-software-using-the-rest-api/verifying-firmware-version-and-upgrade-status.html
Supports: Executing the APIC cluster and switch group upgrades; Fabric discovery through the APIC REST API. Cisco documents three GETs: /api/node/class/firmwareCtrlrRunning.xml for controller firmware, /api/node/class/firmwareRunning.xml for switch firmware, and /api/node/class/maintUpgJob.xml for upgrade status of controllers and switches.

**[12]** Pre-Upgrade/Downgrade Checklists. Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-installation-aci-upgrade-downgrade/Cisco-APIC-Installation-ACI-Upgrade-Downgrade-Guide/m-pre-upgrade-

checklists.html
Supports: Pre-upgrade validation and explained failures; Post-upgrade verification, reporting and rollback decisions. Checklist items include clearing all faults, a configuration export with AES encryption, OOB, CIMC and console access, every APIC Fully Fit (acidiag cluster on 4.2(1)+), all switches Active, standby APIC filesystems below 75%, NTP on APICs and switches, all switch nodes in vPC pairs, APIC disk faults F1527/F1528/F1529 and switch /bootflash faults F1820 (80%) and F1821 (90%), MD5 checks at four points, and a downgrade checklist of features unsupported on older releases.

**[13]** High level summary of APIC upgrades and downgrades. Cisco.
https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-installation-aci-upgrade-downgrade/Cisco-APIC-Installation-ACI-Upgrade-Downgrade-Guide/g-aci-upgrade-architecture/high-level-summary-of-apic-upgrade.html
Supports: Executing the APIC cluster and switch group upgrades. Image installation runs on all APICs in parallel, but database conversion and reload happen one APIC at a time, the next starting only after the previous is back; during conversion DME processes and nginx are down on that APIC, so no disruptive action may be taken; before 6.0(6) the upgrade is complete when all APICs are online and Fully-Fit, and from 6.0(6) the status shows Post Upgrade Pending before Completed.

**[14]** How ACI switch upgrades and downgrades work. Cisco.
https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-installation-aci-upgrade-downgrade/Cisco-APIC-Installation-ACI-Upgrade-Downgrade-Guide/g-aci-upgrade-architecture/high-level-summary-of-switch-upgrade.html
Supports: The Cisco ACI upgrade process; Executing the APIC cluster and switch group upgrades. The APIC pushes the image, the switch checks filesystem and bootflash space for extraction, extracts the image and updates the primary grub partition (the old version moves to the recovery partition), upgrades BIOS/EPLD if needed, does a clean reload and rejoins the fabric; from 2.1(4) an SSD firmware update can cause a second reboot.

**[15]** Guidelines and limitations for upgrading or downgrading. Cisco.
https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-installation-aci-upgrade-downgrade/Cisco-APIC-Installation-ACI-Upgrade-Downgrade-Guide/g-aci-upgrade-architecture/guidelines-and-limitations-for-upgrading-downgrading.html
Supports: Post-upgrade verification, reporting and rollback decisions; Guardrails, approvals and audit for fabric upgrades. If an upgrade stalls or fails, do not reload any APIC, do not decommission a controller and do not switch the firmware target back to the original version; changing the switch firmware group during the upgrade stops the upgrade completing; downgrading APICs from 6.2 or newer to a release older than 6.2(1) is not supported; logs are not accessible while an APIC reboots.

**[16]** Leaf and spine switch upgrades and downgrades through APIC running release 5.1x or later. Cisco.
https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-installation-aci-upgrade-downgrade/Cisco-APIC-Installation-ACI-Upgrade-Downgrade-Guide/g-upgrading-with-apic-release-51-or-later-using-the-gui/upgrading-the-leaf-and-spine-switch-through-apic-running-release-51x-or-later.html
Supports: The Cisco ACI upgrade process; Upgrade planning with structured outputs. In Admin > Firmware > Switches an update group is created, nodes and image selected, Advanced Settings reviewed (Compatibility Check enforced by default, Graceful Upgrade unenforced by default, Run Mode Pause Upon Upgrade Failure / Pause Upon Install Failure / Never Pause), Update starts the pre-download, and Begin Install runs once all nodes show Ready to Install; Done is clicked when all nodes show Completed before moving to the next group.

**[17]** Upgrade or downgrade with APIC release 5.1 or later using the GUI. Cisco.
https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-installation-aci-upgrade-downgrade/Cisco-APIC-Installation-ACI-Upgrade-Downgrade-Guide/g-upgrading-with-apic-release-51-or-later-using-the-gui.html
Supports: Upgrade planning with structured outputs; Guardrails, approvals and audit for fabric upgrades. Starting from release 5.1 the APIC GUI does not provide an option to set a scheduler for the upgrade; pre-download and related benefits are built into the native workflow instead.

**[18]** Cisco APIC Release Notes, Release 6.0(8). Cisco.
https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/6x/release-notes/cisco-apic-release-notes-608.html
Supports: Promo video research; The Cisco ACI upgrade process. Release 6.0(8e) became available on October 28, 2024, followed by 6.0(8f) on December 4, 2024 and 6.0(8h) on June 16, 2025; the companion switch software is Cisco Nexus 9000 ACI-Mode release 16.0(8).

**[19]** Cisco APIC Upgrade/Downgrade Support Matrix. Cisco.
https://www.cisco.com/c/dam/en/us/td/docs/Website/datacenter/apicmatrix/index.html
Supports: The Cisco ACI upgrade process. The matrix takes direction (upgrade or downgrade), a from release and a to release, and returns changes in behavior and open and current-release bugs for APIC and switches; the listed paths have been tested and validated by Cisco, Cisco partners or both.

**[20]** ACI Upgrade Best Practices and Troubleshooting (TAC document 213618). Cisco TAC.
https://www.cisco.com/c/en/us/support/docs/cloud-systems-management/application-policy-infrastructure-controller-apic/213618-aci-upgrade-best-practices-and-troublesh.html
Supports: The Cisco ACI upgrade process; Executing the APIC cluster and switch group upgrades. Put vPC pair leaves and redundant non-vPC leaves such as border leaves in different maintenance groups; keep CIMC at the recommended version and use CIMC KVM if an APIC does not return; check acidiag avread (255 is healthy) before starting; never reboot two or more APICs at the same time; if APIC1 has not finished, do nothing on APIC2 and APIC3; a known failure is APIC2 stuck at 75%.

**[21]** Validations - ACI Pre-Upgrade Validation Script documentation. Cisco (datacenter GitHub org).
https://datacenter.github.io/ACI-Pre-Upgrade-Validation-Script/validations/
Supports: Pre-upgrade validation and explained failures. The /bootflash check compares each switch's available space with what the target release needs (generally twice the target image size) rather than a fixed percentage and reports available versus required space; F1820/F1821 thresholds are 80% and 90%; the standby APIC check runs df -h as rescue-user and alerts at 75%; the NTP check confirms all APICs and switches are synchronized.

**[22]** cisco.aci aci_maintenance_policy module source. Cisco DevNet (ansible-aci).
https://github.com/CiscoDevNet/ansible-aci/blob/master/plugins/modules/aci_maintenance_policy.py
Supports: Guardrails, approvals and audit for fabric upgrades; Executing the APIC cluster and switch group upgrades. The module manages maintMaintP at fabric/maintpol-{name} with options name, run_mode (pauseOnlyOnFailures default, pauseNever, pause_always_between_sets), graceful (default false), scheduler (required for state present, attached through maintRsPolScheduler), ignore_compat, admin_state triggered/untriggered (default untriggered; triggered starts an immediate upgrade), download_state and version.

**[23]** cisco.aci aci_config_snapshot module source. Cisco DevNet (ansible-aci).
https://github.com/CiscoDevNet/ansible-aci/blob/master/plugins/modules/aci_config_snapshot.py
Supports: Firmware staging, backups and the change request. New snapshots are created with the configExportP class and removed with configSnapshot; options include export_policy, max_count (1-10, APIC default 3), include_secure (default true) and format (JSON default); the docs strongly recommend a pause after creating a snapshot and waiting until it finishes before querying, comparing or rolling back.

**[24]** How Claude remembers your project (CLAUDE.md). Anthropic (Claude Code docs).
https://code.claude.com/docs/en/memory
Supports: Claude Code, Agent SDK and MCP setup for the ACI workflow. A project CLAUDE.md lives at ./CLAUDE.md or ./.claude/CLAUDE.md, /init generates a starting file, @path imports pull in other files (up to four hops), files should stay under about 200 lines, and CLAUDE.md is context rather than enforced configuration, so a PreToolUse hook is the way to block an action regardless of what Claude decides.

**[25]** Connect Claude Code to tools via MCP. Anthropic (Claude Code docs).
https://code.claude.com/docs/en/mcp
Supports: Claude Code, Agent SDK and MCP setup for the ACI workflow. Stdio servers are added with claude mcp add [options] <name> -- <command> [args], with --env KEY=value and --scope local|project|user; project scope writes .mcp.json at the repository root for version control, project servers need approval before first use, /mcp shows status and tool counts, and .mcp.json supports ${VAR} and ${VAR:-default} expansion.

**[26]** Get structured output from agents. Anthropic (Claude Agent SDK docs). https://code.claude.com/docs/en/agent-sdk/structured-outputs
Supports: Upgrade planning with structured outputs; Claude Code, Agent SDK and MCP setup for the ACI workflow. Pass output_format={"type": "json_schema", "schema": ...} to query(); a Pydantic model's model_json_schema() can supply the schema; the validated result arrives as structured_output on the ResultMessage, the SDK validates against JSON Schema draft-07 and re-prompts on mismatch, and a run that cannot produce valid output ends with subtype error_max_structured_output_retries.

**[27]** Hooks reference. Anthropic (Claude Code docs). https://code.claude.com/docs/en/hooks
Supports: Guardrails, approvals and audit for fabric upgrades. PreToolUse hooks run before a tool call and can block it with exit code 2 or JSON permissionDecision "deny"; MCP tools are named mcp__<server>__<tool> and matched with patterns such as mcp__memory__.*; hooks live in ~/.claude/settings.json, .claude/settings.json (committed) or .claude/settings.local.json.

**[28]** Configure permissions (Agent SDK). Anthropic (Claude Agent SDK docs). https://code.claude.com/docs/en/agent-sdk/permissions
Supports: Claude Code, Agent SDK and MCP setup for the ACI workflow; Guardrails, approvals and audit for fabric upgrades. The SDK checks hooks, then deny rules, ask rules, permission mode, allow rules and finally the canUseTool callback; disallowed_tools=["Bash"] removes a tool entirely and deny rules apply even in bypassPermissions; allowed_tools does not constrain bypassPermissions; pairing allowed_tools with permission_mode "dontAsk" gives a locked-down headless agent.

**[29]** Model Context Protocol specification: Tools. Model Context Protocol. https://modelcontextprotocol.io/specification/2025-06-18/server/tools
Supports: Claude Code, Agent SDK and MCP setup for the ACI workflow. Clients discover tools with tools/list and invoke them with tools/call; a tool has name, description, inputSchema, optional outputSchema and annotations (which clients must treat as untrusted unless the server is trusted); results can carry structuredContent and isError; there SHOULD always be a human in the loop able to deny invocations, servers MUST validate inputs and rate limit, and clients SHOULD log tool usage for audit.

**[30]** MCP Python SDK. Model Context Protocol. https://github.com/modelcontextprotocol/python-sdk
Supports: Claude Code, Agent SDK and MCP setup for the ACI workflow. Install with uv add "mcp[cli]" or pip install "mcp[cli]" (Python 3.11+); tools are registered with the @mcp.tool() decorator and their schema comes from type hints and docstrings; stdio, Streamable HTTP and SSE transports are supported; uv run mcp dev server.py opens the server in the MCP Inspector.

**[31]** Change Management API. ServiceNow. https://www.servicenow.com/docs/r/api-reference/rest-apis/change-management-api.html
Supports: Firmware staging, backups and the change request. The API creates emergency or normal change requests and standard changes from published templates, processes related approvals, and offers GET /sn_chg_rest/change with sysparm_query, sysparm_offset and order; POST and PATCH need change_manager, itil or admin; one change request is created per call.

**[32]** Change Management state model. ServiceNow. https://www.servicenow.com/docs/r/it-service-management/change-management/c_ChangeStateModel.html
Supports: Firmware staging, backups and the change request; Guardrails, approvals and audit for fabric upgrades. Normal changes move New (-5), Assess (-4, peer and technical approval), Authorize (-3, Change Management and CAB approval), Scheduled (-2, approved and waiting for planned start), Implement (-1), Review (0) and Closed (3); Canceled is 4.

**[33]** Table API. ServiceNow. https://www.servicenow.com/docs/r/api-reference/rest-apis/c_TableAPI.html
Supports: Firmware staging, backups and the change request. The Table API lives at /api/now/table/{tableName}; POST inserts one record and returns 201 with a Location header; GET supports sysparm_query encoded queries (^ for AND, ^OR, ORDERBY/ORDERBYDESC), sysparm_fields, sysparm_limit and sysparm_display_value; PATCH /now/table/{tableName}/{sys_id} updates named fields.

**[34]** Ask for Approval action (Workflow Studio / Flow Designer). ServiceNow. https://www.servicenow.com/docs/r/build-workflows/workflow-studio/ask-approval-flow-designer.html

Supports: Firmware staging, backups and the change request. The Ask for Approval core action takes a Record, a Table with an approval field, Approval Reason, Approval Field, Journal Field, approval Rules and a Due Date that stops a flow waiting forever; its Approval State output is Not Yet Requested, Requested, Approved, Rejected, Cancelled, No Longer Required or Skipped; flow_designer or admin role is needed.

**[35]** Get a development instance (ServiceNow PDI). ServiceNow. https://www.servicenow.com/docs/r/9Lur8HBNLjdzEcjznSAFzA/HTEzwTEvNIUFLxGw~Sq_3A
Supports: Lab environment. A free ServiceNow personal developer instance is obtained by signing in to the ServiceNow Developer Site and following the Personal Developer Instance (PDI) guide under Guides.

**[36]** About protected branches. GitHub Docs. https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches
Supports: Guardrails, approvals and audit for fabric upgrades. Protected branches can require a set number of approving reviews, required status checks that must be successful, skipped or neutral (strict mode also requires the branch to be up to date), code owner review, dismissal of stale approvals when new commits change the diff, and can apply the rules to administrators too.

**[37]** ACI Sandboxes. Cisco DevNet. https://developer.cisco.com/docs/aci/sandbox
Supports: Lab environment; Fabric discovery through the APIC REST API. DevNet offers a shared ACI Simulator (Always-On) with no reservation for APIC API requests, a reservable ACI Simulator sandbox for up to 5 consecutive days, and a Nexus Dashboard Orchestrator sandbox; the sandboxes are zero-cost.

For more info - https://gennoor.com

Gennoor Tech
TRAIN. INNOVATE. BUILD.
For more info - https://gennoor.com